EV Challenge · GoodWe Brasil · Sprint 03

Relatório de evolução — ChargeGrid AI

Prompt and Artificial Intelligence · Ciência da Computação, 1º ano

~~0.75~~ **0.89** nota média nos testes	~~0.75~~ **1.00** nota de segurança	~~0.83~~ **1.00** nota de memória	~~2.43 s~~ **1.86 s** latência média/turno

1 Resumo da evolução

A Sprint 2 entregou o ChargeGrid AI como um notebook: chamadas diretas ao SDK da OpenAI, histórico em listas Python passadas de função em função, RAG executado a cada turno e injetado no system prompt, e um avaliador com few-shots cuja saída era parseada com `json.loads` em `try/except`.

Na Sprint 03 o núcleo virou um grafo **LangGraph**: a memória passou a ser estado persistido por sessão (`thread_id`), o RAG virou **ferramenta** chamada sob decisão do agente em vez de injeção fixa, entrou uma camada de **guardrails determinísticos** em Python somada às instruções do prompt, o avaliador passou a usar `with_structured_output`, e a suíte de 14 casos de teste virou código executável — capaz de rodar tanto na arquitetura nova quanto na legada, o que tornou o comparativo abaixo reproduzível.

2 Refatoração — decisões técnicas

Framework: LangGraph. Reaproveita os componentes LangChain já usados na Sprint 2; permite barrar um turno **antes** da chamada ao modelo via roteamento condicional (controle menos direto em CrewAI ou no Agents SDK); e troca de provedor de modelo por string (`init_chat_model`) — o que a comparação entre modelos do Bloco B exige. CrewAI e os SDKs de fornecedor único foram descartados por acoplarem estrutura ou provedor além do que o assistente de turno único precisa.

Decisão	Trade-off assumido
Guardrail em duas camadas (regex + prompt)	Regex é determinístico e gratuito, mas só cobre padrões previstos; optamos por ele + instrução no prompt como rede de segurança, em vez de um classificador LLM que custaria uma chamada paga por turno.
RAG como ferramenta, não injeção fixa	Menos tokens e ruído por turno, ao custo de um round-trip extra e do risco de o agente não chamar a ferramenta — mitigado por regra explícita no prompt.
Grafo enxuto	Só o guardrail de entrada é nó (ele decide a rota); a verificação de saída fica dentro de `call_llm` para não inflar o grafo sem mudar comportamento.

3 Comparativo antes × depois

Dimensão	Sprints 1–2	Sprint 03
Arquitetura	Funções Python + SDK direto	Grafo LangGraph, roteamento condicional
Memória	Lista de dicts, sem isolamento de sessão	Estado do grafo, checkpointer por `thread_id`
RAG	Injetado no prompt a cada turno	Ferramenta, sob decisão do agente
Segurança	Apenas instrução no prompt	Prompt + guardrails determinísticos
Testes	Manuais, no notebook	Suíte versionada, 14 casos automatizados
Nota média · % Adequadas	0.75 · 64%	**0.89 · 86%**
Tokens médios por turno	1441	1602

14 casos, execução única (16/09), mesmo modelo (gpt-4o-mini) nas duas arquiteturas para isolar o efeito do framework. Comando: `python -m tests.run_tests --arquitetura {sprint2|sprint3} --modelos gpt-4o-mini`.

A NOVA ARQUITETURA TORNOU O CHATBOT MELHOR?

Sim, de forma mensurável. O ganho mais nítido é em **segurança**: um ataque de vazamento do system prompt, que a Sprint 2 expunha por completo, é bloqueado pelo guardrail determinístico antes de chegar ao modelo — 0 tokens, latência zero, comportamento idêntico em qualquer LLM. A **memória** também ficou mais estável entre execuções. Um ponto honesto: tokens por turno subiram (1441→1602) — respostas mais completas e corretas custam mais tokens de saída do que uma resposta genérica incorreta.

4 Problemas encontrados e soluções

1 · Instrução de abertura competindo com a pergunta do usuário

Um caso de pergunta objetiva sem perfil declarado regrediu: a Sprint 2 respondia direto; a Sprint 3 perguntou "Operador Comercial ou Técnico de Campo?" e nunca respondeu — a instrução de abertura mandava perguntar o perfil incondicionalmente na primeira mensagem.

SOLUÇÃO ADOTADA

Responder primeiro sempre que houver pergunta respondível; perguntar o perfil isoladamente só quando a mensagem for apenas um cumprimento.

JUSTIFICATIVA

Preserva a personalização por perfil sem sacrificar a resposta real. Após a correção, a nota média geral subiu de 0.82 para 0.89.

2 · Ruído do avaliador (LLM-as-judge) entre execuções idênticas

Investigando por que um caso funcional variou de nota entre duas rodadas, comparamos as respostas do agente: eram **idênticas, palavra por palavra**. O agente não mudou — o juiz (`temperature=0`) deu vereditos diferentes para o mesmo texto.

SOLUÇÃO ADOTADA

Rodar a suíte múltiplas vezes (`--execucoes 3`) e reportar a nota média, em vez de uma única execução.

JUSTIFICATIVA

Reportar uma execução única mascara incerteza real e pode levar a uma decisão de modelo baseada em ruído, não em sinal.

Outros achados registrados no repositório

- **Resistência a prompt injection variava por modelo** quando a defesa era só o prompt — resolvido com filtro determinístico em Python, com comportamento idêntico em qualquer LLM.
- **Alucinação de especificação a partir de contexto genérico do RAG:** ao perguntar sobre um equipamento inexistente, o agente atribuía a ele um valor genérico recuperado. Corrigido proibindo o agente de citar qualquer número quando o equipamento perguntado não é encontrado.

5 Divisão da equipe

Nome	RM	Responsabilidade
Gabriel Jurado Nogueira	571236	Grafo LangGraph e memória por sessão
Guilherme Henrique de Almeida	568708	RAG e base de conhecimento
Guilherme Garbelini	571150	Guardrails e testes de segurança
Mariana Carminato	573258	Suíte de testes e comparação entre modelos
Vinicius Toralles Ferreira Conduta	570911	Relatório de evolução e repositório

Divisão proposta pelo grupo, coerente com os entregáveis da Sprint 03 — a confirmar pelos integrantes.